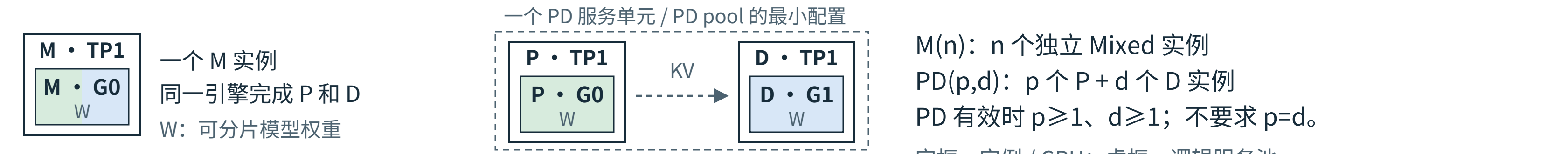


Role change | TP1 的完整变换图

同一模型 · PP=1 · 每个引擎实例恰好占 1 张 GPU；双向箭头表示一对互逆变换。

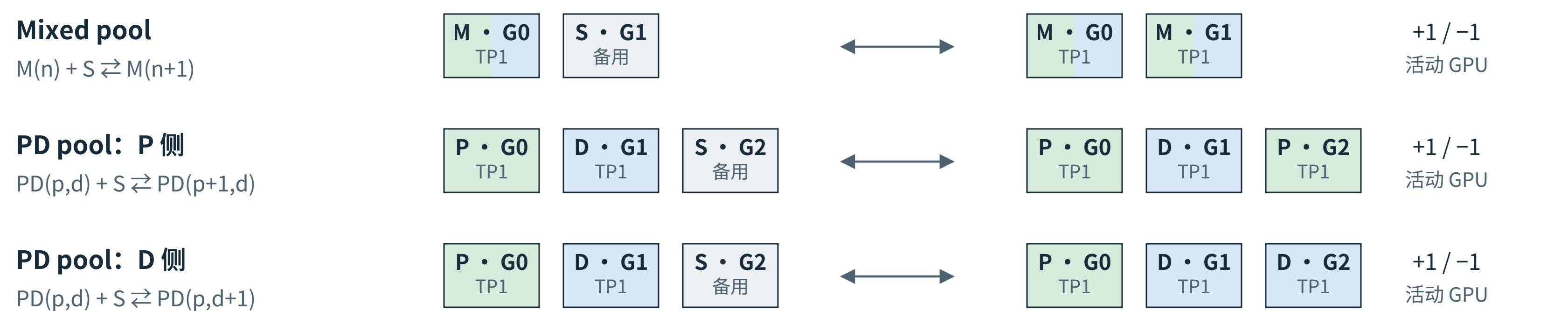


A 先固定单位：PD 是服务路径，P、D 分别是实例



B add / delete GPU：增加或减少单卡副本

向右 add；向左 delete



TP change | PD 与 Mixed 的 TP 增减

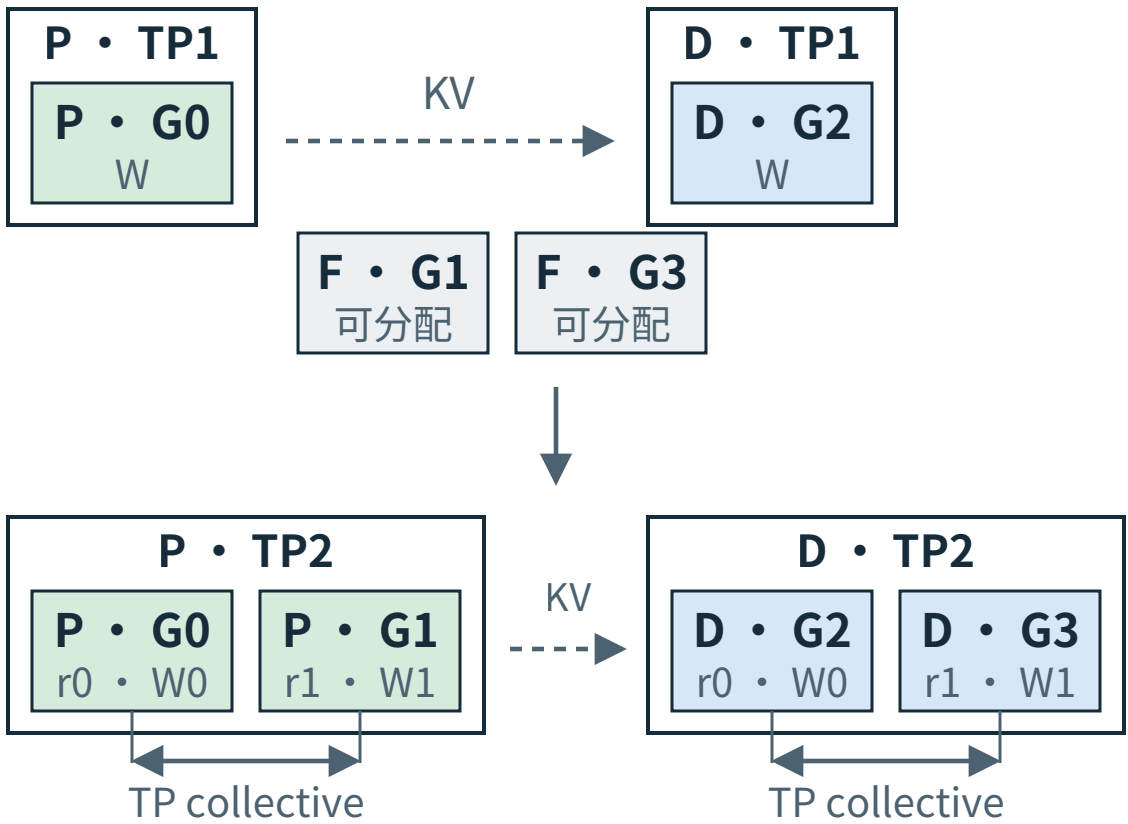
目标重建机制：当前 slow_reshard_tp 运行入口尚未支持；以下为结构与步骤示意。



主四图保持引擎数量不变：从空闲池取卡 / 向空闲池还卡。例子均为同模型、PP=1。

1 PD 两端的 TP 增加

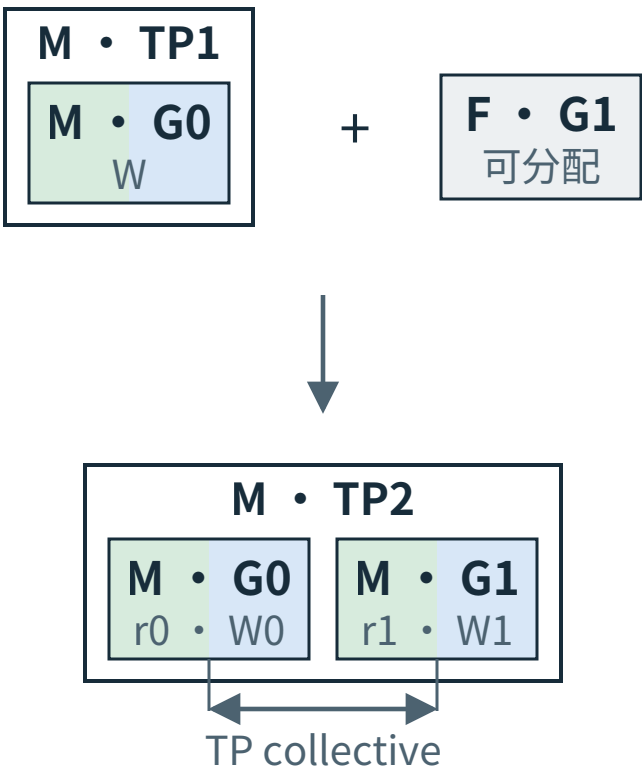
TP1 → TP2



活动 GPU：2 → 4；P、D 仍各 1 个引擎。

2 Mixed 实例的 TP 增加

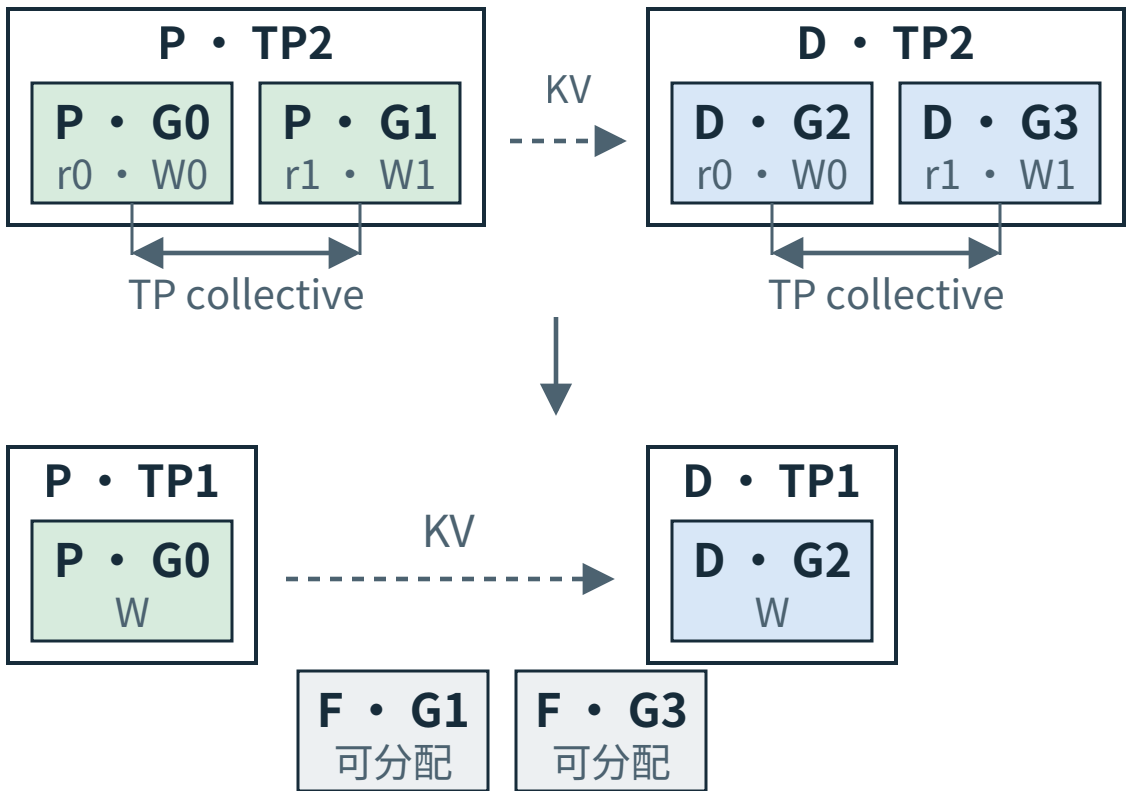
TP1 → TP2



活动 GPU：1 → 2；仍是 1 个 Mixed 引擎。

3 PD 两端的 TP 减少

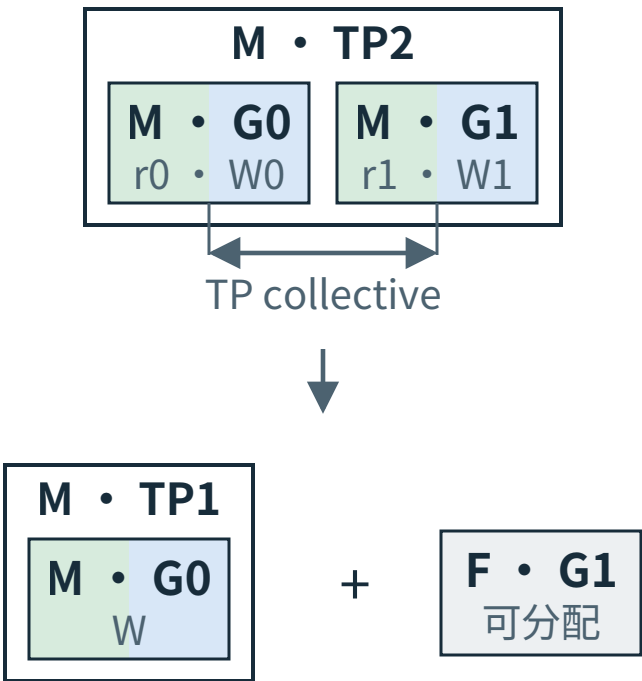
TP2 → TP1



活动 GPU：4 → 2；G1、G3 归还资源池。

4 Mixed 实例的 TP 减少

TP2 → TP1



活动 GPU：2 → 1；G1 归还资源池。

A 一般式：所有增减方向与单侧 P / D 变换

$R(t) + (u-t)F \rightleftharpoons R(u)$

$R \in \{P, D, M\}$ ； $u > t$ ；一个引擎占用 $t \rightarrow u$ 张 GPU。

$PD(t) + 2(u-t)F \rightleftharpoons PD(u)$

$PD(t) = P(t) + D(t)$ ；双侧对称扩缩。

$1 \rightleftharpoons 2, 2 \rightleftharpoons 4, 1 \rightleftharpoons 4$ 同理；目标必须满足模型和显存约束。

只改变 P 或只改变 D 的 TP

$P(t) \rightarrow P(u)$ ：新请求必须配已就绪的 $D(u)$ 。

$D(t) \rightarrow D(u)$ ：新请求必须配已就绪的 $P(u)$ 。

增加和减少均适用；原对端可重建或换成兼容对端。

当前不支持 $P(1) \rightarrow D(2)$ 或 $P(2) \rightarrow D(1)$ 接力。

配对还要求 model / PP / pool / generation 一致。

B 固定 GPU 数的实例 merge / split

副本数变化；不是主四图中的取卡 / 还卡



$2 \times M(TP1) \rightleftharpoons 1 \times M(TP2)$

$4M(1) \rightleftharpoons 2M(2) \rightleftharpoons M(4)$ ；反向 split 需加载完整副本。

$2P(1) + 2D(1) \rightleftharpoons P(2) + D(2)$

P 侧单独合并、D 侧单独合并；共 4 张 GPU。

合并后是 P、D 两个 TP2 组，不是一个 TP4。

通式： $n \times R(t) \rightleftharpoons m \times R(u)$ ， $n \cdot t = m \cdot u$ 。

保持单实例时缩 TP 释放卡；另建完整副本才是 split。

C 所有 TP 变换共用的重建流程



排空屏障：所有旧 rank 的请求、活跃 KV、pending transfer 均为 0；旧 TP 的活跃 KV 不迁入新 TP。

重建：workers / TP communicator → 加载已验证权重分片 → 空 KV 池 → 输出与取消 / KV 清理探针。

失败：保持关闭，重建并验证旧布局；恢复不确定则隔离 GPU。其他实例只能在自身容量内继续服务。

权重与通信 W 为可分片权重；TP2： $W \rightarrow \{W0, W1\}$ 。P、D 各自模型完整；KV 与组内 collective 不同。

每卡显存 可分片权重约 W/t + 复制张量 + KV + workspace + 通信 / 加载峰值；缩 TP 前必须装得下。

边界 $TP \in \{1, 2, 4\}$ 、PP=1 还需模型/profile 校验；切换已有不同 TP 常驻池，仅改变路由，不等于重建 TP。

TP2 KV 对应：P/r0 → D/r0，P/r1 → D/r1；目标布局发布后，仅新请求使用这条兼容 KV 路径。